实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**数据库系统**			学期	**2025 年秋季学期**
任课教师	刘艳艳	完成日期	2026/1/14	上机课时间	2026/1/7
实 验 名 称	SQL Server 数据库备份、恢复技术实验				

一、实验要求（10%）

对实验二所设计的数据库，完成如下实验要求：

1、通过创建关系实验体会关系数据理论，函数依赖以及范式

（1）设计一个关系，满足二范式而不满足三范式

（2）设计一个关系，满足三范式而不满足 BC 范式，要求详细描述关系的语义，分析关系中存在的函数依赖。

（3）使用 MySQL 或 SQL Server 设计实现，录入数据。（可使用华为 openGauss 数据库）

（4）体会数据冗余、增加异常、删除异常、修改复杂。

2、在学生表新增加一个字段 AvgGrade，存放学生的平均成绩。创建触发器，录入或者修改学生成绩时触发，自动计算学生平均成绩，并修改 AvgGrade 的值

3、数据库备份和恢复

（1）对实验二所涉及的数据库进行备份，包括数据文件和日志文件的备份，要求至少使用两种备份方法，如海量备份和增量备份

（2）利用备份对数据库的恢复操作，理解事务故障恢复、系统故障恢复以及介质故障恢复原理

（3）通过备份和恢复，理解数据库关于事务、日志、备份和恢复等概念。理解一个数据库 DBMS 事务处理的原理和应用。

二、实验内容及步骤（80%）

（一）创建关系体会关系数据理论，函数依赖以及范式

1、设计一个关系，满足二范式而不满足三范式

（1）2NF 和 3NF

- 2NF：每一个非主属性完全函数依赖于码（消除非主属性对码的部分函数依赖）
- 3NF：每一个非主属性即不部份依赖于码，也不传递依赖于码（消除非主属性对码的传递函数依赖）

（2）创建关系

- 新关系：student_instructor 学生-辅导员表
- 语义：一个学生属于一个班级，一个班级只有一个辅导员
- 属性：学号、班级号、辅导员姓名。

  student_instructor(stu_id, cs_id, instructor)
- 外键：stu_id，cs_id

（3）函数依赖分析

- stu_id → cs_id（完全依赖）
- cs_id → instructor（班级决定辅导员）

满足 2NF：候选码为单属性 stu_id，所以不存在非主属性对码的部份依赖

不满足 3NF：stu_id → cs_id 且 cs_id → instructor，所以存在传递依赖 stu_id → instructor

（4）建表实现与数据录入

```sql
DROP TABLE IF EXISTS student_instructor;
CREATE TABLE student_instructor
(
    stu_id      int             COMMENT '学号'    PRIMARY KEY ,
    cs_id       int             COMMENT '班级号',
    instructor  varchar(20)     COMMENT '辅导员姓名',
    FOREIGN KEY student_instructor(stu_id) REFERENCES student(stu_id),
    FOREIGN KEY student_instructor(cs_id) REFERENCES class(cs_id)
)COMMENT '学生-辅导员 2NF';

INSERT INTO student_instructor
VALUES (1,1,'王小菲'),
       (2,2,'乔英');
```

2、设计一个关系，满足三范式而不满足 BC 范式

（1）BCNF

- BCNF：每一个决定属性因素都包含码（消除主属性对码的部分和传递依赖

（2）创建关系

- 新关系：teacher_project_guide 教师项目指导表
- 语义：学生参加科研项目，一个院系一个项目，一个项目可以有多个老师，一名老师只负责一个项目，一名学生在一个项目中只能由一位老师来指导
- 属性：学号、教师号、项目院系（用院系代替项目）

  teacher_project_guide（stu_id, tc_id, col_id)
- 外键：stu_id, tc_id, col_id

（3）函数依赖分析

(stu_id, col_id) → tc_id（完全依赖）

tc_id → col_id（老师决定项目）

满足 3NF：候选码为（stu_id, col_id），每一个非主属性即不部份依赖于码，也不传递依赖于码

不满足 BCNF：tc_id 为决定因素，而决定因素不全是候选码

（4）建表实现与数据录入

```
DROP TABLE IF EXISTS teacher_project_guide;
CREATE TABLE teacher_project_guide
(
    stu_id  int     NOT NULL PRIMARY KEY,
    tc_id   int     NOT NULL,
    col_id  int     NOT NULL,
    FOREIGN KEY (stu_id) REFERENCES student(stu_id),
    FOREIGN KEY (tc_id) REFERENCES teacher(tc_id),
    FOREIGN KEY (col_id) REFERENCES college(col_id)
) COMMENT '3NF非BCNF实验表：决定因素不全是候选码';

INSERT INTO teacher_project_guide
VALUES (1, 1, 3);
```

3、体会数据冗余、增加异常、删除异常、修改复杂

（1）数据冗余

在学生-辅导员表中，同一个班级（cs_id）的每个学生记录里都要存储一遍辅导员的名字。如果有 1000 个学生，名字存 1000 遍。

	stu_id	cs_id	instructor
1	2	2	乔英
2	1	1	王小菲
3	4	1	王小菲

（2）插入异常

3 号老师（王武）负责 1 号学院（信工学部）的项目，但还没学生参加。

```
education_system> INSERT INTO teacher_project_guide (tc_id, col_id) VALUES (3, 1)
[2026-01-14 18:16:27] [HY000][1364] Field 'stu_id' doesn't have a default value
```

出现报错如图，由于 stu_id 存在 not null 的约束条件，因此不能再没有学生的情况下，单独录入“某老师负责某项目”的对应关系。

（3）删除异常

参加 3 号学院项目的唯一学生（stu_id = 1）退学了。

```
DELETE FROM teacher_project_guide WHERE stu_id = 1;
```

虽然代码运行成功了，但项目指导表中，关于“1 号老师负责 3 号学院项目”的这一教学安排信息也被删除了。

（4）修改异常

班级 cs_id = 1 的辅导员换人了，需要遍历并修改该班级下所有学生的记录，否则会出现数据不一致（有的学生对应旧辅导员，有的对应新辅导员）。

（二）使用触发器

1、在学生表新增加一个字段 AvgGrade，存放学生的平均成绩。

在 student 表中添加一个 AvgGrade 字段

```
ALTER TABLE student ADD COLUMN AvgGrade DECIMAL(4,1)
    DEFAULT 0.0 COMMENT '平均成绩';
```

默认值为 0.0

	stu_id	stu_name	gender	col_id	cs_id	AvgGrade
1	1	秦英	1	2	1	0.0
2	2	晓晓	1	3	2	0.0
3	3	大王	3	4	3	0.0
4	4	小王	2	2	1	0.0

将 take_course 中的成绩数据同步到 AvgGrade 字段

	tcrs_id	stu_id	crs_id	grade
1	1	1	1	91.0
2	2	1	2	89.5
3	3	2	3	73.2

	stu_id	stu_name	gender	col_id	cs_id	AvgGrade
1	1	秦英	1	2	1	90.3
2	2	晓晓	1	3	2	73.2
3	3	大王	3	4	3	0.0
4	4	小王	2	2	1	0.0

2、创建触发器

针对 take_course 选课表创建触发器，平均分的改变是由该表中的数据变动引起的处理两种情况：

- 录入成绩时（AFTER INSERT）：新增一门课成绩后，重算平均分。
- 修改成绩时（AFTER UPDATE）：某门课成绩改了，重算平均分。

在 MySQL 中，同一个触发器不能同时监听 INSERT 和 UPDATE，所以需要写两个。

```
# 录入成绩时的触发器
DELIMITER //
CREATE TRIGGER trg_after_insert_grade
AFTER INSERT ON take_course
FOR EACH ROW
BEGIN
    UPDATE student
    SET AvgGrade = (SELECT AVG(grade) FROM take_course
                                      WHERE stu_id = NEW.stu_id)
    WHERE stu_id = NEW.stu_id;
END //
```

```
# 修改成绩时的触发器
CREATE TRIGGER trg_after_update_grade
AFTER UPDATE ON take_course
FOR EACH ROW
BEGIN
    -- 只有当成绩字段发生变化时才更新，节省开销
    IF OLD.grade <> NEW.grade THEN
        UPDATE student
        SET AvgGrade = (SELECT AVG(grade) FROM take_course
                                          WHERE stu_id = NEW.stu_id)
        WHERE stu_id = NEW.stu_id;
    END IF;
END //
DELIMITER ;
```

3、触发器运行

录入或者修改学生成绩时触发，自动计算学生平均成绩，并修改 AvgGrade 的值

（1）为学号为 1 的同学录入新的成绩

```
# 给学号为1的学生加一门课成绩
INSERT INTO take_course (tcrs_id, stu_id, crs_id, grade)
VALUES (4, 1, 3, 60.0);
```

结果：

	stu_id	stu_name	gender	col_id	cs_id	AvgGrade
1	1	秦英	1	2	1	90.3

	stu_id	stu_name	gender	col_id	cs_id	AvgGrade
1	1	秦英	1	2	1	80.2

（2）修改成绩

```
# 修改学号为1的学生的成绩
UPDATE take_course SET grade = 98.0 WHERE tcrs_id = 4;
```

	stu_id	stu_name	gender	col_id	cs_id	AvgGrade
1	1	秦英	1	2	1	92.8

（三）数据库备份和恢复

1、对实验二所涉及的数据库进行备份，包括数据文件和日志文件的备份，要求至少使用两种备份方法，如海量备份和增量备份

（1）开启二进制日志

增量备份依赖于 MySQL 的 Binary Log (binlog)。如果没开启，无法做增量备份。

检查是否开启：

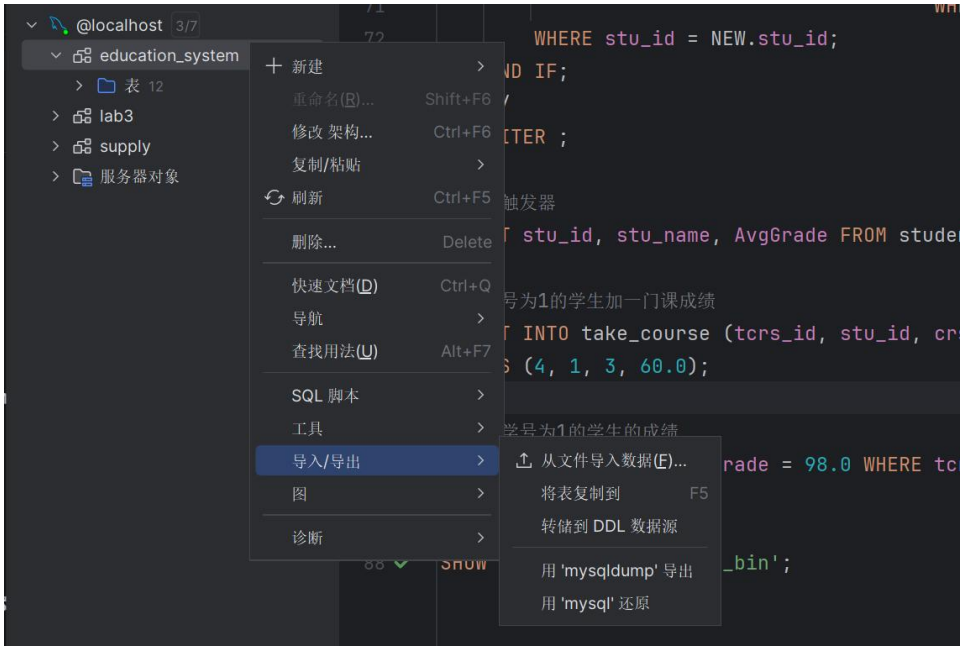
```
SHOW VARIABLES LIKE 'log_bin';
```

	Variable_name	Value
1	log_bin	ON

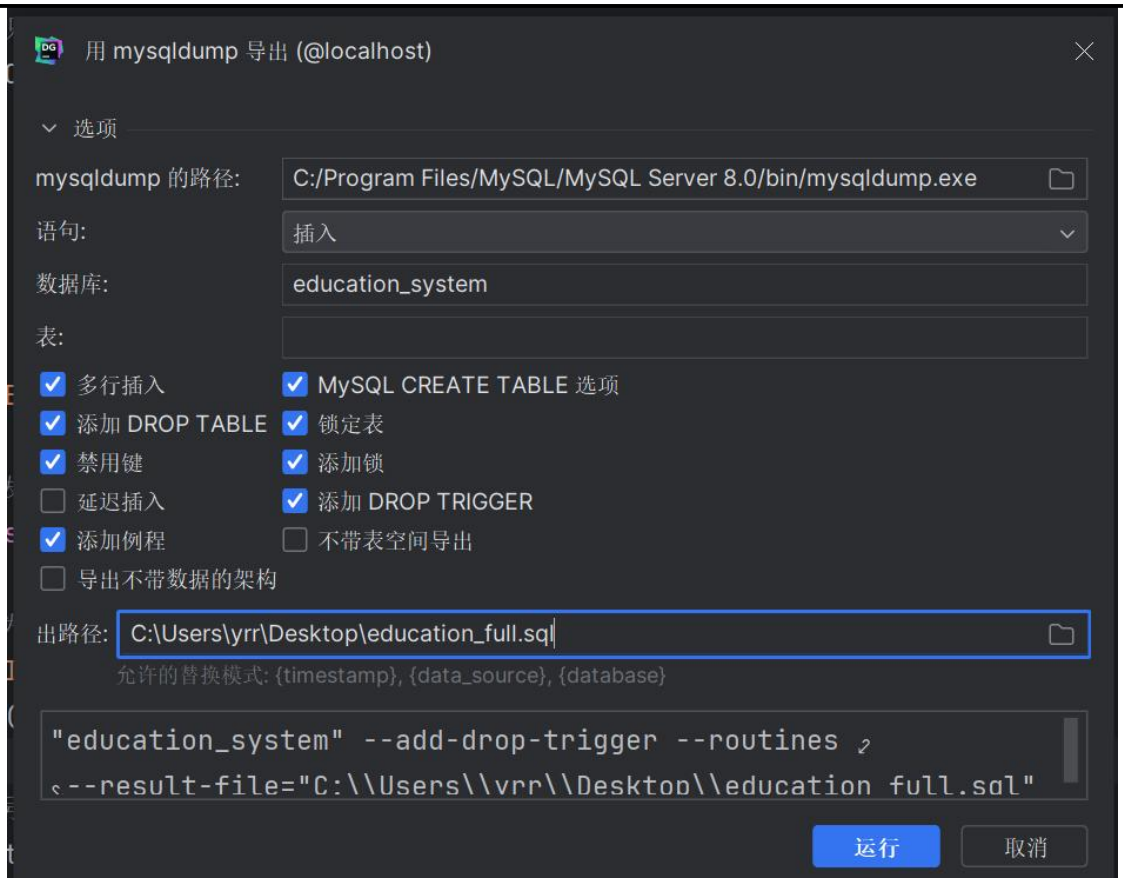
（2）海量备份

- 海量备份：备份当前时刻数据库的全部数据

导出数据，选择用 mysqldump 导出



配置导出文件，确定 mysqldump 路径和导出文件路径：







（3）增量备份

只备份自上次备份以来发生变化的数据。在 MySQL 中，这表现为备份 binlog 文件。

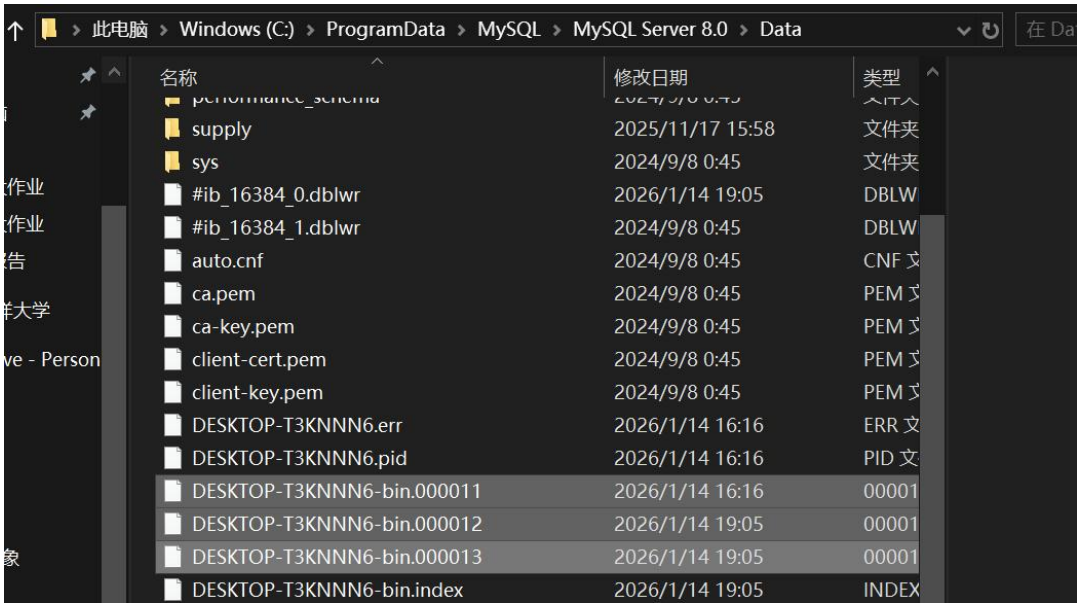
随便修改一条数据：98 → 96

```
# 修改学号为1的学生的成绩
UPDATE take_course SET grade = 96.0 WHERE tcrs_id = 4;
```

刷新日志：强制 MySQL 关闭当前的 binlog 文件并开启一个新的

```
# 刷新日志
FLUSH LOGS;
```

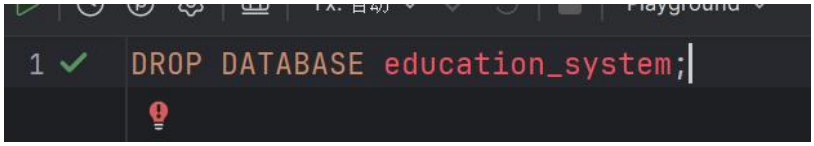
在电脑文件夹中找到 MySQL 的二进制日志（Binlog），其中 000013 是最新的一个，即增量备份。



2、利用备份对数据库的恢复操作，理解事务故障恢复、系统故障恢复以及介质故障恢复原理

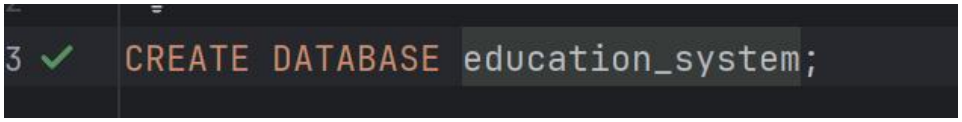
（1）模拟介质故障

模拟数据库被误删

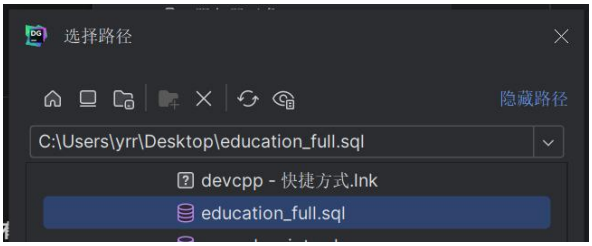


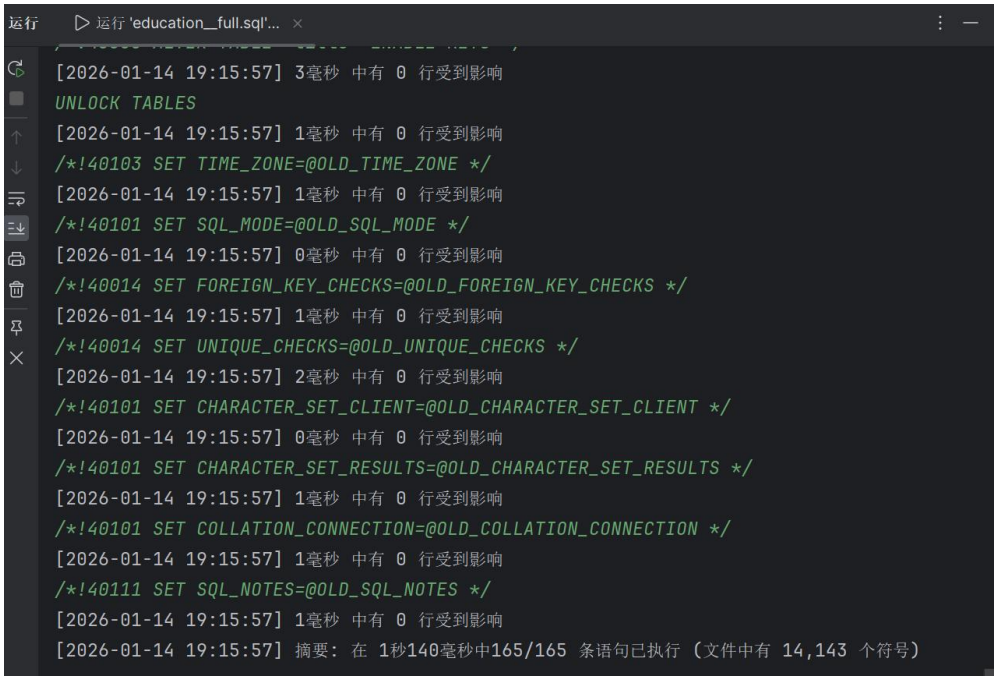
- 恢复海量

新建一个同名空库，右键点击该库 -> 运行 SQL 脚本 (Run SQL Script) -> 选择海量备份的文件



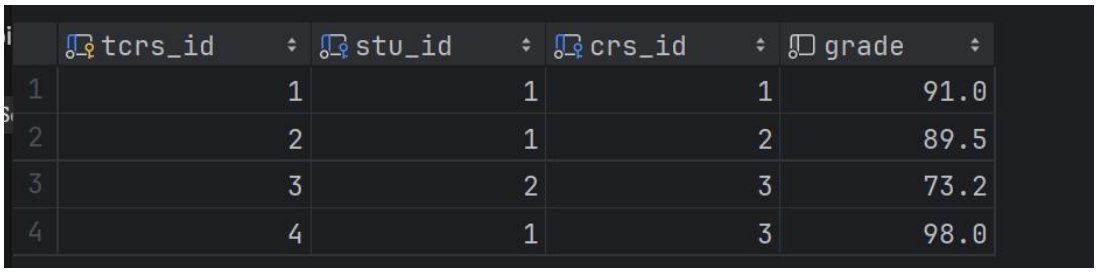






成功恢复到数据备份的那一刻：

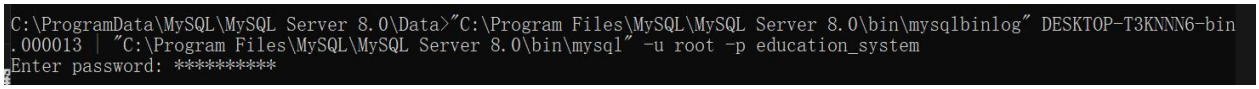
在进行增量备份所修改的数据之前：



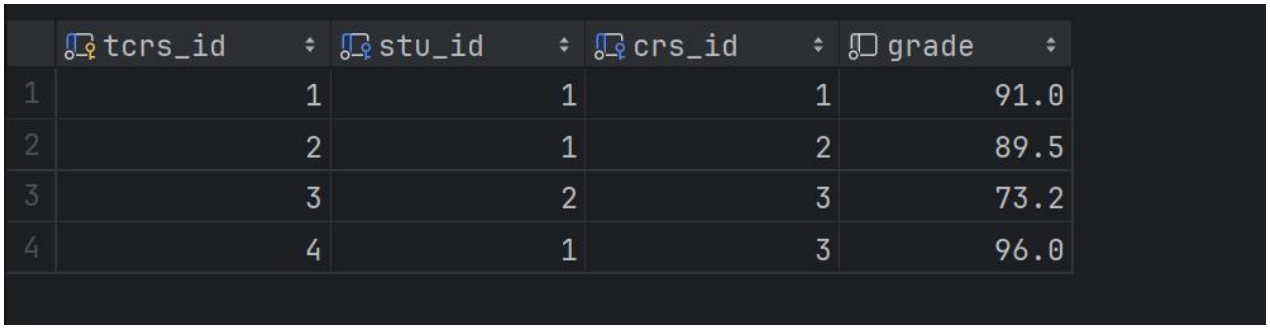
	tcrs_id	stu_id	crs_id	grade
1	1	1	1	91.0
2	2	1	2	89.5
3	3	2	3	73.2
4	4	1	3	98.0

- 恢复增量

由于 binlog 是二进制文件，不能直接执行。需要打开命令行终端（CMD 或 PowerShell），使用 MySQL 自带的工具：



98 → 96



	tcrs_id	stu_id	crs_id	grade
1	1	1	1	91.0
2	2	1	2	89.5
3	3	2	3	73.2
4	4	1	3	96.0

（2）理解三种故障恢复

- 事务故障恢复

场景：程序运行到一半崩溃，或事务执行了 ROLLBACK。

原理：DBMS 利用 Undo Log（回滚日志）。将该事务已完成的操作撤销，恢复到事务开始前

的状态（原子性）。

```
# 事务故障
START TRANSACTION;
UPDATE student SET stu_name = '错误的名字' WHERE stu_id = 1;
SELECT * FROM student;
```

stu_id	stu_name	gender	col_id	cs_id
1	错误的名字	1	2	
2	晓晓	1	3	

回滚：

```
ROLLBACK ;
```

	stu_id	stu_name	gender	col_id	cs_id
1	1	秦英	1	2	1
2	2	晓晓	1	3	2

- 系统故障恢复

场景：正在写数据时突然断电。内存中的数据（Buffer Pool）没来得及写入磁盘。

原理：DBMS 重启时，利用 Redo Log（重做日志）。将已提交但未写入磁盘的数据“重做”一遍，保证提交的数据不丢失（持久性）。

- 介质故障恢复

场景：硬盘坏了，数据彻底没了。

原理：利用外部备份（海量备份 + 增量备份/binlog）。先重装数据库并导入海量，再按顺序重放 binlog（即刚才在命令行做的操作）。

3、通过备份和恢复，理解数据库关于事务、日志、备份和恢复等概念。理解一个数据库 DBMS 事务处理的原理和应用

（1）事务

事务是 DBMS 执行的最小逻辑单位。它必须满足 ACID 特性。

结合实验：

当向 take_course 录入成绩时，触发器会自动更新 student 表的 AvgGrade。

- 原子性 Atomicity：插入成绩和更新平均分必须“同生共死”。如果平均分更新失败，插入成绩的操作也必须撤回。
- 持久性 Durability：一旦点击“提交”或执行成功，即使下一秒电脑断电，数据也必须保存在硬盘里。

（2）日志

日志是事务处理的核心。MySQL 实际上有三种关键日志：

- Undo Log（回滚日志）：记录“反向操作”（比如 INSERT 一条，记录一条 DELETE）。用于事务故障时的 ROLLBACK。

• Redo Log（重做日志）：最核心的性能设计。数据库为了快，先写日志到磁盘（顺序写很快），再慢慢改数据文件。断电重启后，DBMS 靠它把没写完的数据补回来。

• Binlog（二进制日志）：即操作的 DESKTOP...bin.000013。它不用于系统崩溃重启，而是用于介质故障恢复和主从复制。

（3）备份

• 海量备份：

手段：mysqldump。

本质：它是数据库在某一时间点的“静态快照”。

缺点：大数据库导出极慢，且备份后的变动无法涵盖。

• 增量备份：

手段：保存 Binlog。

本质：它记录的是“动态过程”（即从快照点开始的所有 SQL 变动）。

优点：占用空间小，通过“全备+增量”可以实现 PIRT（点对点恢复），即恢复到过去任意一个秒钟。

（4）恢复

• 事务故障恢复：靠 Undo Log（自动）。

• 系统故障恢复：靠 Redo Log（重启时自动）。

• 介质故障恢复：靠 人工干预（实验操作）：

三、心得总结（写出自己在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

（一）遇到的问题

1、在进行增量备份恢复时，使用命令行输入命令，一直显示另一个程序站在使用此文件，进程无法访问，关闭 dataGrip 之后重新运行依旧出现如此报错：

```
C:\ProgramData\MySQL\MySQL Server 8.0\Data>DESKTOP-T3KNNN6-bin.000013 | "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql" -u root -p education_system
另一个程序正在使用此文件，进程无法访问。
```

通过查询资料,得知文件被占用可能是指要恢复的日志文件目前很可能是 MySQL 正在使用的活跃日志文件，MySQL 正在往里写数据，所以系统锁定了它，不让其他程序（比如你的命令行）读取。

解决方法：通过在查询控制台运行命令 FLUSH LOGS; MySQL 会关闭当前的 .000013 文件，并创建一个新的 .000014 来继续记录。这样 .000013 就变成了“历史文件”，锁定会被解除，就可以读取它了。

```
C:\ProgramData\MySQL\MySQL Server 8.0\Data>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlbinlog" DESKTOP-T3KNNN6-bin.000013 | "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql" -u root -p education_system
Enter password: **********
```

（二）体会与收获

通过本次实验，我系统实践了从数据库规范化设计、自动化维护到安全保障的全流程。在关系理论实验中，通过对比 2NF 与 BCNF，我直观体会到了范式设计在消除数据冗余与操作异

常中的核心作用；触发器的应用则让我感受到了 DBMS 维护数据一致性的自动化优势。

最令我受益匪浅的是备份恢复实操：我不仅掌握了 mysqldump 海量恢复与 Binlog 增量恢复的技术细节，更由表及里地理解了事务 ACID 特性、Redo/Undo 日志以及介质故障恢复的深层原理。我深刻认识到，数据库并非简单的存储容器，而是依靠严密的日志机制与事务逻辑支撑起数据安全的复杂系统。这次实验不仅夯实了我的实操技能，更重塑了我对 DBMS 底层运行机制的认知。